Buổi 7

Đa năng hóa toán tử (phần 1)

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Nguyên tắc cơ bản

1.1 Đa năng hóa toán tử là gì?
1.2 Các giới hạn của đa năng hóa toán tử
1.3 Trình biên dịch dịch biểu thức có toán tử như thế nào?

2. Hàm toán tử là thành viên hay không là thành viên

2.1 Hai cách cài đặt hàm toán tử
2.2 Vì sao toán tử đối xứng nên là friend?
2.3 Chuyển kiểu ngầm qua constructor và từ khóa explicit

3. Đa năng hóa toán tử hai ngôi

3.1 Các toán tử số học hai ngôi
3.2 Toán tử so sánh và toán tử gán kết hợp
3.3 Ví dụ: toán tử cho lớp Time

4. Đa năng hóa toán tử một ngôi

4.1 Toán tử một ngôi: đổi dấu, phủ định
4.2 Toán tử ++ và -- dạng tiền tố và hậu tố
4.3 Các lỗi thường gặp khi đa năng hóa toán tử

5. Ví dụ tổng hợp và thực hành

5.1 Lớp PhanSo đầy đủ và kết quả chạy
5.2 Tóm tắt và bài thực hành buổi 7
5.3 Câu hỏi và bài tập

1.1 Đa năng hóa toán tử là gì?

- **Vấn đề:** với lớp PhanSo (buổi 4) ta phải viết `c = a.Cong(b.Nhan(d));` – khó đọc. Ta muốn viết **tự nhiên** như với kiểu cơ sở: `c = a + b * d;`

Đa năng hóa toán tử (operator overloading)

Định nghĩa lại ý nghĩa của các toán tử có sẵn (+, −, *, ==, <<, [], …) khi toán hạng là **đối tượng của lớp** do người dùng định nghĩa. Cài đặt bằng một hàm có tên đặc biệt `operator@` (@ là ký hiệu toán tử).

```
class PhanSo
{  public:
      PhanSo(int t = 0, int m = 1);
      PhanSo operator+(const PhanSo &b) const;  // a + b
      ...
   private:
      int Tu, Mau;
};
PhanSo PhanSo::operator+(const PhanSo &b) const
{  return PhanSo(Tu * b.Mau + b.Tu * Mau, Mau * b.Mau);  }
PhanSo a(1, 2), b(1, 3), c;
c = a + b;            // trình biên dịch dịch thành:
c = a.operator+(b);   // hai cách viết tương đương
c.In();               // 5/6
```

- **C++ đã đa năng hóa sẵn** nhiều toán tử cho kiểu cơ sở: `+` với int, double; `<<` vừa là dịch bit vừa là xuất dữ liệu (`cout << x`); `*` vừa là nhân vừa là giải tham chiếu con trỏ.
- **Lợi ích:** mã nguồn ngắn gọn, gần với ký hiệu toán học; lớp do người dùng định nghĩa được dùng **như kiểu có sẵn** (string `+`, `==`; complex; ma trận…).
- **Bản chất:** chỉ là **cú pháp thuận tiện** cho lời gọi hàm – mọi việc làm được bằng toán tử đều làm được bằng hàm thường.

1.2 Các giới hạn của đa năng hóa toán tử

- **Được đa năng hóa:**

```
+   -   *   /   %   ^   &   |   ~   !
=   <   >   +=  -=  *=  /=  %=  ^=  &=
|=  <<  >>  >>= <<= ==  !=  <=  >=  &&
||  ++  --  ->* ,   ->  []  ()  new delete
new[]  delete[]
```

- **KHÔNG được đa năng hóa:**

Toán tử	Ý nghĩa
`.`	Truy cập thành viên
`.*`	Truy cập thành viên qua con trỏ thành viên
`::`	Phạm vi
`?:`	Điều kiện ba ngôi
`sizeof`	Kích thước

- **Các quy tắc phải tuân thủ:**
  - **Không** tạo toán tử mới (không có `**` để lũy thừa).
  - **Không** thay đổi **độ ưu tiên**, **tính kết hợp**, **số ngôi** của toán tử: `a + b * c` vẫn tính `*` trước; `!` vẫn là một ngôi.
  - **Không** định nghĩa lại toán tử cho **kiểu cơ sở**: ít nhất một toán hạng phải là đối tượng của lớp (hoặc enum).
  - Không có tham số mặc định cho hàm toán tử (trừ `()`).
  - Đa năng hóa `+` **không** tự động sinh ra `+=`; phải định nghĩa riêng.

Ví dụ 7.1 – Thông báo lỗi của g++

```
int operator+(int a, int b);
// error: 'int operator+(int, int)' must have an
//        argument of class or enumerated type
A operator.(A a);
// error: expected type-specifier before '.' token
```

- **Ngoại lệ:** `=` và `&` (lấy địa chỉ) được trình biên dịch tự sinh cho mọi lớp; `=` mặc định sao chép từng thành viên (buổi 5).

1.3 Trình biên dịch dịch biểu thức có toán tử như thế nào?

- Mỗi toán tử trong biểu thức được thay bằng **một lời gọi hàm toán tử**, theo đúng **độ ưu tiên** và **tính kết hợp** của toán tử gốc.

Biểu thức	Được dịch thành (toán tử là thành viên)
`a + b`	`a.operator+(b)`
`a + b * c`	`a.operator+(b.operator*(c))`
`a + b + c`	`(a.operator+(b)).operator+(c)` – kết hợp trái
`-a`	`a.operator-()`
`a == b`	`a.operator==(b)`
`++a` / `a++`	`a.operator++()` / `a.operator++(0)`
`a = b + c`	`a.operator=(b.operator+(c))`

Ví dụ 7.0 – Phân tích `PhanSo d = a + b * PhanSo(3);`

1. Tạo đối tượng tạm `T1 = PhanSo(3)` ⇒ 3/1.
2. `T2 = b.operator*(T1)` – nhân **trước** vì `*` ưu tiên hơn `+`.
3. `T3 = a.operator+(T2)`.
4. d được khởi tạo từ T3; các đối tượng tạm bị hủy cuối câu lệnh.

- **Hệ quả:**
  - Muốn biết một biểu thức gọi những hàm nào ⇒ viết lại dưới dạng lời gọi hàm.
  - Toán tử `&&`, `||`, `,` đa năng hóa **mất** tính "đánh giá tắt" (cả hai vế luôn được tính) ⇒ hầu như không nên đa năng hóa.
- **Nguyên tắc vàng:** toán tử phải có **ý nghĩa tự nhiên**: `+` của PhanSo là phép cộng phân số; dùng `+` để "in" hay `-` để "cộng" làm mã rất khó hiểu.

2.1 Hai cách cài đặt hàm toán tử

Cách	Toán tử hai ngôi `a @ b`	Toán tử một ngôi `@a`	Số tham số tường minh
Hàm thành viên	`a.operator@(b)`	`a.operator@()`	Hai ngôi: 1; một ngôi: 0 (toán hạng trái là `*this`)
Hàm không thành viên (thường là friend)	`operator@(a, b)`	`operator@(a)`	Hai ngôi: 2; một ngôi: 1

```
// Cách 1 – hàm thành viên
class PhanSo
{  public:
      PhanSo operator+(const PhanSo &b) const
      {  return PhanSo(Tu * b.Mau + b.Tu * Mau,
                       Mau * b.Mau);
      }
   ...
};
```

```
// Cách 2 – hàm friend (không thành viên)
class PhanSo
{  friend PhanSo operator+(const PhanSo &a,
                           const PhanSo &b);
   ...
};
PhanSo operator+(const PhanSo &a, const PhanSo &b)
{  return PhanSo(a.Tu * b.Mau + b.Tu * a.Mau,
                 a.Mau * b.Mau);
}
```

- **Quy tắc lựa chọn:**

Toán tử	Nên cài đặt là
`=`, `[]`, `()`, `->`	**Bắt buộc** thành viên
`+=`, `-=`, `++`, `--`, `-` một ngôi	Thành viên (thay đổi/thuộc về đối tượng)
`+`, `-`, `*`, `/`, `==`, `<` (đối xứng)	Không thành viên (friend) – để chuyển kiểu cho **cả hai** toán hạng
`<<`, `>>` với cout/cin	**Bắt buộc** không thành viên (toán hạng trái là ostream/istream – buổi 8)

- Hàm thành viên: toán hạng trái **phải** là đối tượng của lớp.
- Hàm friend: truy cập được private mà vẫn nhận hai toán hạng bình đẳng.

2.2 Vì sao toán tử đối xứng nên là friend?

Ví dụ 7.2 – Phép cộng phân số với số nguyên

- Constructor `PhanSo(int t = 0, int m = 1)` cho phép **chuyển ngầm** `int → PhanSo` (2 ⇒ 2/1).
- **Nếu operator+ là hàm thành viên:**

```
PhanSo a(1, 2), c;
c = a + 2;   // a.operator+(PhanSo(2)) ⇒ 5/2  ✓
c = 2 + a;   // 2.operator+(a) ⇒ LỖI!
```

```
error: no match for 'operator+'
       (operand types are 'int' and 'P')
```

- Toán hạng trái `2` là int, không phải đối tượng ⇒ không có hàm thành viên nào để gọi.

- **Nếu operator+ là friend (hai tham số):**

```
c = a + 2;   // operator+(a, PhanSo(2))  ✓
c = 2 + a;   // operator+(PhanSo(2), a)  ✓
```

- Cả hai toán hạng đều được chuyển kiểu ⇒ phép toán **đối xứng** như toán học.
- **Cách khác:** viết thêm các hàm friend riêng:

```
friend PhanSo operator*(int k, const PhanSo &a)
{  return PhanSo(k * a.Tu, a.Mau);  }
c = 2 * a;   // 1/1
```

Kết quả chạy thử (lớp PhanSo đầy đủ ở mục 5)

`c = a + 2;` ⇒ **5/2**; `c = 2 * a;` ⇒ **1/1**; với `operator+` là thành viên, `2 + a` **không biên dịch được**.

2.3 Chuyển kiểu ngầm qua constructor và từ khóa explicit

- Constructor **một tham số** (hoặc các tham số sau có giá trị mặc định) đóng vai trò **toán tử chuyển kiểu** từ kiểu tham số sang lớp:

```
PhanSo(int t = 0, int m = 1);
PhanSo a(1, 2), c;
c = a + 2;        // 2 ⇒ PhanSo(2, 1) tự động
c = 5;            // c = PhanSo(5)
void In(PhanSo p);
In(7);            // In(PhanSo(7))
```

- **Tiện lợi** nhưng đôi khi gây chuyển kiểu **ngoài ý muốn**: lớp `Mang(int n)` (buổi 5) cho phép viết `Mang m = 10;` hay truyền số 10 vào hàm nhận Mang – rất khó hiểu.

- **explicit** cấm chuyển kiểu ngầm, chỉ cho phép gọi constructor **tường minh**:

```
class PS
{  public:
      explicit PS(int t, int m = 1) : T(t), M(m) {}
      friend PS operator+(const PS &a, const PS &b);
      ...
};
PS p(1, 2);
PS q = p + PS(1);   // tường minh ⇒ 3/2 ✓
PS r = p + 1;       // LỖI
```

```
error: no match for 'operator+'
       (operand types are 'PS' and 'int')
```

- **Khuyến nghị:** dùng `explicit` cho constructor một tham số **không** mang ý nghĩa "chuyển đổi tự nhiên" (Mang(int n), String(int dungLuong)); bỏ explicit khi chuyển đổi là tự nhiên (int → PhanSo, double → Complex).
- Chiều ngược lại (PhanSo → double) dùng **toán tử chuyển kiểu** `operator double()` – buổi 8.

```
class PhanSo
{  public:
      PhanSo(int t = 0, int m = 1)
      {  Tu = t;  Mau = (m != 0) ? m : 1;  RutGon();  }
      PhanSo operator+(const PhanSo &b) const
      {  return PhanSo(Tu * b.Mau + b.Tu * Mau, Mau * b.Mau);
}
      PhanSo operator-(const PhanSo &b) const
      {  return PhanSo(Tu * b.Mau - b.Tu * Mau, Mau * b.Mau);
}
      PhanSo operator*(const PhanSo &b) const
      {  return PhanSo(Tu * b.Tu, Mau * b.Mau);  }
      PhanSo operator/(const PhanSo &b) const
      {  return PhanSo(Tu * b.Mau, Mau * b.Tu);  }
      void In() const { cout << Tu << "/" << Mau; }
   private:
      int Tu, Mau;
      void RutGon();
};
```

- **Các điểm thiết kế quan trọng:**
  - Tham số `const PhanSo &b`: không sao chép, không sửa b.
  - Hàm `const`: phép cộng **không** thay đổi a (a + b không làm a thay đổi).
  - Trả về **giá trị** `PhanSo` (đối tượng mới), **không** trả về tham chiếu – kết quả là đối tượng tạm, trả tham chiếu tới biến cục bộ là lỗi (buổi 3).
  - Tạo kết quả bằng constructor ⇒ tự rút gọn, tự kiểm tra mẫu $\neq 0$.

Ví dụ 7.3 – Chạy thử với a = 1/2, b = 1/3

```
c = a + b;  c.In();   // 5/6
(a - b).In();         // 1/6
(a * b).In();         // 1/6
(a / b).In();         // 3/2
PhanSo d = a + b * PhanSo(3);   // * trước + ⇒ 1/2 + 1 =
3/2
```

- Độ ưu tiên giữ nguyên như toán học: `b * PhanSo(3)` được tính **trước**.

3.2 Toán tử so sánh và toán tử gán kết hợp

- **Toán tử so sánh** trả về `bool`:

```
bool operator==(const PhanSo &b) const
{  return Tu == b.Tu && Mau == b.Mau;  }   // nhờ luôn tối
giản
bool operator!=(const PhanSo &b) const
{  return !(*this == b);  }               // dùng lại ==
bool operator<(const PhanSo &b) const
{  return Tu * b.Mau < b.Tu * Mau;  }     // Mau > 0
bool operator>(const PhanSo &b) const
{  return b < *this;  }
```

```
cout << (a == PhanSo(2, 4)) << " " << (b < a);  // 1 1
```

- `PhanSo(2, 4)` được rút gọn thành 1/2 ⇒ bằng a. Nếu lớp **không** giữ bất biến "tối giản", `==` phải so sánh chéo `Tu*b.Mau == b.Tu*Mau`.
- **Mẹo:** chỉ cài đặt `==` và `<` rồi suy ra `!=`, `>`, `<=`, `>=` để tránh mâu thuẫn.

- **Toán tử gán kết hợp** `+=`, `-=`… **sửa** toán hạng trái ⇒ hàm thành viên, **không** const, trả về `PhanSo &` (để viết `a += b += c`):

```
PhanSo &operator+=(const PhanSo &b)
{  *this = *this + b;      // dùng lại operator+
   return *this;
}
a += b;  a.In();           // 1/2 + 1/3 = 5/6
```

- **Cách viết hiệu quả hơn** (thư viện chuẩn hay dùng): cài `+=` trước, rồi định nghĩa `+` qua `+=`:

```
friend PhanSo operator+(PhanSo a, const PhanSo &b)
{  a += b;  return a;  }   // a là bản sao
```

Toán tử	Thành viên?	const?	Trả về
`a + b`	Friend/thành viên	Có	Giá trị mới
`a += b`	Thành viên	Không	`PhanSo &` (*this)
`a == b`	Friend/thành viên	Có	`bool`

3.3 Ví dụ: toán tử cho lớp Time

- Toán hạng **không nhất thiết** cùng kiểu và kết quả **không nhất thiết** là đối tượng của lớp – chọn theo ý nghĩa thực tế.

```
class Time
{  public:
      Time(int h = 0, int m = 0, int s = 0)
      {  Giay = (h * 3600L + m * 60 + s) % 86400;  }
      Time operator+(long n) const      // Time + số giây
      {  return Time(0, 0, (Giay + n % 86400 + 86400) %
86400);  }
      long operator-(const Time &t) const   // Time − Time
      {  return Giay - t.Giay;  }        // ⇒ số giây
      bool operator<(const Time &t) const
      {  return Giay < t.Giay;  }
      Time &operator++()                // thêm 1 giây
      {  Giay = (Giay + 1) % 86400;  return *this;  }
      void In() const;                  // HH:MM:SS
   private:
      long Giay;                        // 0 .. 86399
};
```

Ví dụ 7.6 – Chạy thử

```
Time a(8, 30), b(10, 15, 20);
(a + 3600).In();  (b + 45).In();
cout << b - a << " " << (a < b);
Time c(23, 59, 59);  ++c;  c.In();
```

```
09:30:00 10:16:05
6320 1
00:00:00
```

Biểu thức	Kiểu toán hạng	Kiểu kết quả
`a + 3600`	Time + long	Time
`b - a`	Time − Time	long (số giây)
`a < b`	Time < Time	bool

- `b − a` = 10:15:20 − 08:30:00 = 1 giờ 45 phút 20 giây = **6320** giây.
- `++c` từ 23:59:59 quay vòng về **00:00:00**.

4.1 Toán tử một ngôi: đổi dấu, phủ định

- Toán tử một ngôi `@a`:
  - **Thành viên:** không có tham số tường minh – `a.operator@()`.
  - **Không thành viên:** một tham số – `operator@(a)`.

```
// đổi dấu: -a
PhanSo operator-() const
{  return PhanSo(-Tu, Mau);  }
// phủ định: !a đúng khi a = 0
bool operator!() const
{  return Tu == 0;  }
PhanSo a(1, 2);
(-a).In();                // -1/2
if (!PhanSo(0, 5)) cout << "bang 0";
```

- **Phân biệt** `-` một ngôi và hai ngôi theo **số tham số**: `operator-()` (một ngôi, thành viên) và `operator-(const PhanSo &)` (hai ngôi, thành viên) cùng tồn tại – đa năng hóa.

Ví dụ 7.4 – Lớp Complex (số phức) theo giáo trình

```
class Complex
{  public:
      Complex(double r = 0, double i = 0)
      {  Real = r;  Imag = i;  }
      Complex operator+(const Complex &z) const
      {  return Complex(Real + z.Real, Imag + z.Imag);  }
      Complex operator*(const Complex &z) const
      {  return Complex(Real * z.Real - Imag * z.Imag,
                        Real * z.Imag + Imag * z.Real);  }
      Complex operator-() const          // một ngôi
      {  return Complex(-Real, -Imag);  }
      Complex operator~() const          // số phức liên
hợp
      {  return Complex(Real, -Imag);  }
      void Print() const
      {  cout << "(" << Real << ", " << Imag << ")";  }
   private:
      double Real, Imag;
};
Complex x(4.3, 8.2), y(3.3, 1.1);
(x + y).Print();     // (7.6, 9.3)
(x * ~y).Print();    // (23.21, 22.33)
```

4.2 Toán tử ++ và -- dạng tiền tố và hậu tố

Phân biệt tiền tố và hậu tố

- **Tiền tố** `++a`: tăng a **rồi** trả về **chính a** ⇒ `PhanSo &operator++();`
- **Hậu tố** `a++`: trả về **giá trị cũ** của a, rồi mới tăng ⇒ `PhanSo operator++(int);` – tham số **int giả** (không dùng, chỉ để phân biệt).

```
// ++ tiền tố: cộng 1 (= Mau/Mau)
PhanSo &operator++()
{  Tu += Mau;
   return *this;           // trả về chính đối tượng
}
// ++ hậu tố
PhanSo operator++(int)     // int: tham số giả
{  PhanSo cu = *this;      // lưu giá trị cũ
   Tu += Mau;              // tăng
   return cu;              // trả về GIÁ TRỊ cũ
}
```

Ví dụ 7.5 – Chạy thử

```
PhanSo x(1, 2), y;
y = ++x;   x.In(); cout << " "; y.In();   // 3/2 3/2
y = x++;   x.In(); cout << " "; y.In();   // 5/2 3/2
```

- `++x`: x thành 3/2, y nhận 3/2 (giá trị **mới**).
- `x++`: x thành 5/2, y nhận 3/2 (giá trị **cũ**).

- **Hiệu năng:** hậu tố phải tạo **bản sao** `cu` và trả về theo giá trị ⇒ chậm hơn tiền tố. Với đối tượng (iterator…), nên viết `++it` thay vì `it++` khi không cần giá trị cũ.
- **Vì sao tiền tố trả `&` còn hậu tố trả giá trị?** Tiền tố trả về chính đối tượng (có thể `++++x`); hậu tố trả về giá trị cũ là **biến cục bộ** `cu` ⇒ không được trả tham chiếu.
- `--` tiền tố/hậu tố cài đặt tương tự.

4.3 Các lỗi thường gặp khi đa năng hóa toán tử

Lỗi	Ví dụ	Hậu quả
operator+ **sửa** toán hạng trái	`PhanSo operator+(const PhanSo &b) { Tu += …; return *this; }`	`c = a + b` làm **a bị thay đổi** – sai ngữ nghĩa
Trả về tham chiếu tới biến cục bộ	`PhanSo &operator+(…) { PhanSo kq(…); return kq; }`	Tham chiếu treo – kết quả rác
Quên `const` cho hàm toán tử chỉ đọc	`bool operator==(const PhanSo &b);`	Không so sánh được hai đối tượng hằng / `const PhanSo &`
Quên tham số giả `int` của hậu tố	Định nghĩa hai lần `operator++()`	Lỗi định nghĩa lại; không có `a++`
Hậu tố trả về `*this` sau khi tăng	`PhanSo operator++(int) { Tu += Mau; return *this; }`	`y = x++` cho giá trị **mới** (sai)
`+=` trả về giá trị thay vì tham chiếu	`PhanSo operator+=(…)`	`(a += b) += c` không cộng c vào a
Mâu thuẫn giữa các toán tử	Viết riêng `<` và `>` với hai công thức khác nhau	`a < b` và `b > a` cho kết quả khác nhau

Ví dụ 7.7 – Tìm lỗi

```
PhanSo PhanSo::operator-(const PhanSo &b)
{  Tu = Tu * b.Mau - b.Tu * Mau;
   Mau = Mau * b.Mau;
   return *this;
}
```

→ Hàm **sửa chính đối tượng** gọi (a bị thay đổi sau `c = a - b`), thiếu `const`, thiếu rút gọn. **Sửa:** `PhanSo operator-(const PhanSo &b) const { return PhanSo(Tu * b.Mau - b.Tu * Mau, Mau * b.Mau); }`.

5.1 Lớp PhanSo đầy đủ và kết quả chạy

```
class PhanSo
{  public:
      PhanSo(int t = 0, int m = 1);
      PhanSo operator+(const PhanSo &b) const;
      PhanSo operator-(const PhanSo &b) const;
      PhanSo operator*(const PhanSo &b) const;
      PhanSo operator/(const PhanSo &b) const;
      bool operator==(const PhanSo &b) const;
      bool operator<(const PhanSo &b) const;
      PhanSo operator-() const;
      PhanSo &operator++();
      PhanSo operator++(int);
      PhanSo &operator+=(const PhanSo &b);
      friend PhanSo operator*(int k, const PhanSo &a);
      void In() const;
   private:
      int Tu, Mau;
      void RutGon();
};
```

```
int main()
{  PhanSo a(1, 2), b(1, 3), c;
   c = a + b;  c.In();                    // 5/6
   (a - b).In(); (a * b).In(); (a / b).In();
   c = a + 2;  c.In();                    // ngầm int→PhanSo
   c = 2 * a;  c.In();                    // friend
   (-a).In();
   cout << (a == PhanSo(2, 4)) << " " << (b < a);
   PhanSo x(1, 2), y;
   y = ++x;  x.In(); y.In();
   y = x++;  x.In(); y.In();
   a += b;  a.In();
   PhanSo d = a + b * PhanSo(3);  d.In();
}
```

```
5/6  1/6  1/6  3/2
5/2  1/1  -1/2
1 1
3/2 3/2  |  5/2 3/2
5/6
11/6
```

- Dòng cuối: a đã thành 5/6 sau `a += b`; `b * 3` = 1 ⇒ 5/6 + 1 = **11/6**.

5.2 Tóm tắt và bài thực hành buổi 7

Toán tử	Khai báo khuyến nghị
`a + b`, `a - b`, `a * b`, `a / b`	`friend PhanSo operator+(const PhanSo &, const PhanSo &);` hoặc thành viên `const`
`a == b`, `a < b`	`bool operator==(const PhanSo &) const;`
`a += b`	`PhanSo &operator+=(const PhanSo &);`
`-a`, `!a`	`PhanSo operator-() const;`
`++a`	`PhanSo &operator++();`
`a++`	`PhanSo operator++(int);`

Bài thực hành – Lớp PhanSo với +, −, *, /, ==, ++

- 1) Cài đặt lớp PhanSo như mục 5.1 (constructor có rút gọn và chuẩn hóa dấu).
- 2) Cài đặt các toán tử `+ − * /` dạng **friend**; kiểm thử `a + 2`, `2 + a`, `3 * a`, `a / 0` (xử lý chia cho phân số 0: in cảnh báo và trả về a).
- 3) Cài đặt `== != < > <= >=` (chỉ viết trực tiếp `==` và `<`); sắp xếp mảng 5 phân số bằng Bubble Sort dùng toán tử `<`.
- 4) Cài đặt `+= -= *= /=`, `-` một ngôi, `++`, `--` tiền tố và hậu tố; viết đoạn kiểm thử in rõ giá trị trước/sau.
- 5) Tính tổng S = 1/1 + 1/2 + … + 1/n bằng lớp PhanSo với n = 10 (đáp số **7381/2520**).

5.3 Câu hỏi và bài tập

Câu hỏi

- 1) Đa năng hóa toán tử là gì? Nêu các toán tử không đa năng hóa được và các quy tắc phải tuân thủ.
- 2) So sánh cài đặt toán tử bằng hàm thành viên và hàm friend. Vì sao `2 + a` không biên dịch được khi `operator+` là hàm thành viên?
- 3) Phân biệt khai báo `++` tiền tố và hậu tố. Vì sao hậu tố trả về giá trị còn tiền tố trả về tham chiếu?
- 4) Vì sao `operator+` trả về đối tượng (giá trị) còn `operator+=` trả về tham chiếu?

Bài tập dự đoán kết quả (có đáp số)

```
PhanSo a(3, 4), b(1, 4);
PhanSo c = a + b * 2;   c.In(); cout << " ";
PhanSo d = a++ + ++b;   d.In(); cout << " ";
a.In(); cout << " "; b.In();
```

→ **Đáp số:** `5/4 2/1 7/4 5/4` – `b * 2` = 1/2 ⇒ c = 5/4; `a++` cho 3/4 (a thành 7/4), `++b` cho 5/4 ⇒ d = 3/4 + 5/4 = 2/1.

Bài tập lập trình

- 1) Lớp **Complex**: `+ - *`, `==`, `-` một ngôi, `~` liên hợp, hàm `Modul()`; giải phương trình bậc hai có nghiệm phức.
- 2) Lớp **Vector2D** (x, y): `+`, `-`, nhân vô hướng `k * v` và `v * k`, tích vô hướng `u * v` (trả về double), `==`.
- 3) Lớp **Time**: `t + n` (cộng n giây), `t2 - t1` (số giây chênh lệch), `++t` (tăng 1 giây), `<`.